

CS 486/686

Neural Networks II: Training

Yuntian Deng

Lecture 20

RN 21.1–21.2 · PM 7.5 · GBC 6.5, 7, 8

Search ›

Uncertainty ›

Decisions ›

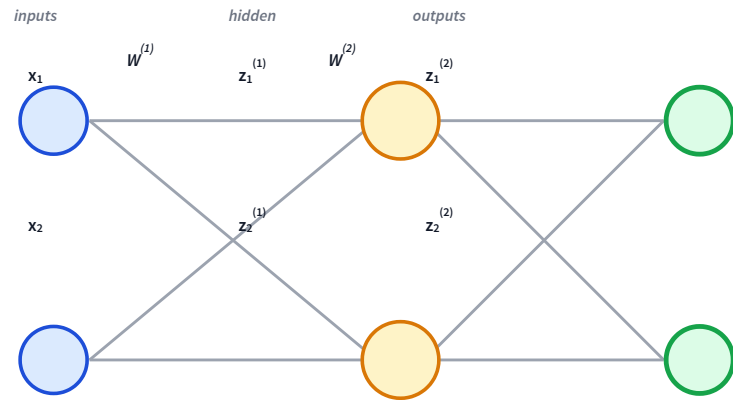
Learning

Learning goals

- Write the **cross-entropy** loss and the **gradient-descent** update.
- Describe **back-propagation** (forward + backward passes) and compute a weight's gradient.
- Speed up training with **momentum** and **Adam / AdamW**.
- Fight overfitting with **regularization, dropout, and normalization**.
- Decide when to use a **neural network** vs a **decision tree**.

The training problem

A 2-layer feedforward network for spam classification — predict $[a_1^{(2)}, a_2^{(2)}]$ from two features (email length, sender trust).



Softmax outputs a probability over "spam" vs "ham":

- Spam $\rightarrow$ one-hot target $[1, 0]$
- Ham $\rightarrow$ one-hot target $[0, 1]$

We have paired data (x_1, x_2, y) . How do we set the weights $W^{(1)}, W^{(2)}$?

Loss function and gradient descent

For classification we use the **cross-entropy** loss from L16 (softmax output $\hat{y}^{(i)}$, one-hot target $y^{(i)}$):

$$L(W) = - \sum_{i=1}^N \sum_k y_k^{(i)} \log \hat{y}_k^{(i)}$$

(For regression, swap in squared error $\frac{1}{2} \sum_i \|\hat{y}^{(i)} - y^{(i)}\|^2$.)

Gradient descent: repeat for many iterations:

1. Compute the gradient $\nabla_W L(W) = [\partial L / \partial w]_{w \in W}$.

2. Update every weight: $w \leftarrow w - \eta \frac{\partial L}{\partial w}$.

$\eta > 0$ is the *learning rate* (step size).

Why this works

Direction: steepest descent

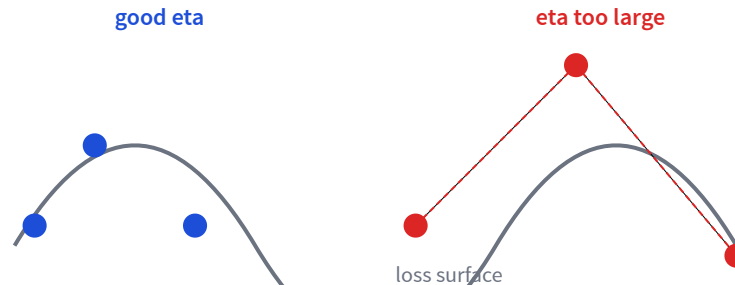
$\nabla_W L$ points in the direction of *fastest increase* of L .

Moving in $-\nabla_W L$ decreases L the fastest.

Step size: η trade-off

Too small $\Rightarrow$ very slow learning.

Too large $\Rightarrow$ overshoots the minimum, may diverge.



Three flavors of gradient descent

Batch GD

all N examples

Gradient uses **all** N examples per step.

Stable, accurate gradient.

Slow for large N .

Stochastic GD

1

One example per step.

Cheap; noise helps escape local minima.

Noisy updates.

Mini-batch GD

batch of 32

A small batch (e.g. 32, 64) per step.

Best of both worlds; GPU-friendly.

Standard choice today.

First, how do we *get* the gradient efficiently? Then how do we take smarter steps (momentum, Adam)?

Back-propagation: efficient gradients

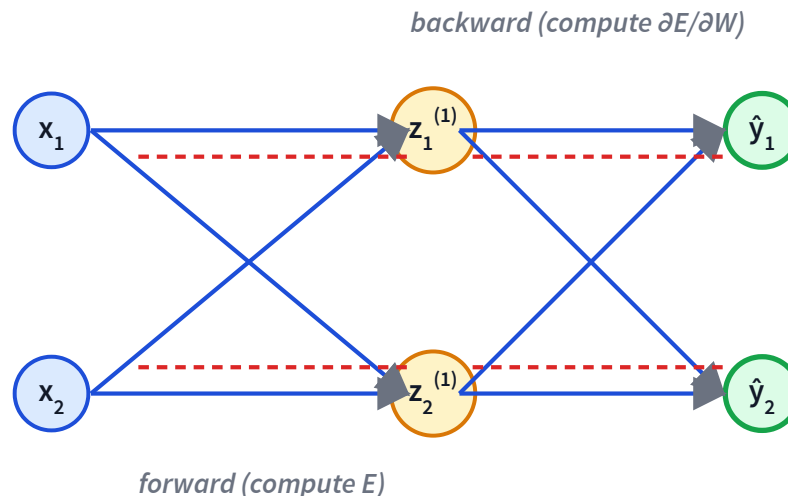
Two passes per training example (x, y) :

→ Forward pass

Push inputs through layers, compute every $a^{(\ell)}$, $z^{(\ell)}$, and the loss E .

← Backward pass

Propagate ∂E back through the layers, compute every $\partial E / \partial W^{(\ell)}$.



Forward pass

Compute pre-activations $a^{(\ell)}$ and activations $z^{(\ell)} = g(a^{(\ell)})$ layer by layer.

$$a_j^{(1)} = \sum_i x_i W_{i,j}^{(1)}, \quad z_j^{(1)} = g(a_j^{(1)})$$

$$a_k^{(2)} = \sum_j z_j^{(1)} W_{j,k}^{(2)}, \quad z_k^{(2)} = g(a_k^{(2)})$$

$$E = E(z^{(2)}, y)$$

Cache every $a^{(\ell)}$ and $z^{(\ell)}$ — we'll need them on the way back.

Backward pass

Walk back through the network, computing local errors δ via the chain rule.

Layer 2 (output side):

$$\frac{\partial E}{\partial W_{j,k}^{(2)}} = \delta_k^{(2)} z_j^{(1)}, \quad \delta_k^{(2)} = \frac{\partial E}{\partial z_k^{(2)}} g'(a_k^{(2)})$$

Layer 1 (propagate back):

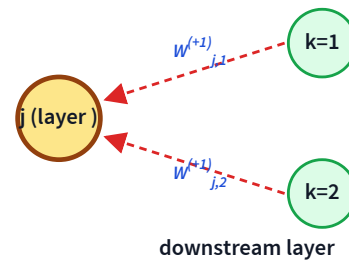
$$\frac{\partial E}{\partial W_{i,j}^{(1)}} = \delta_j^{(1)} x_i, \quad \delta_j^{(1)} = \left(\sum_k \delta_k^{(2)} W_{j,k}^{(2)} \right) g'(a_j^{(1)})$$

Pattern: gradient = local error $\times$ input to that weight.

The recursive structure of δ

For unit j in layer ℓ , define $\delta_j^{(\ell)} = \frac{\partial E}{\partial a_j^{(\ell)}}$. Then:

$$\delta_j^{(\ell)} = \begin{cases} \frac{\partial E}{\partial z_j^{(\ell)}} g'(a_j^{(\ell)}), & \text{output unit (base)} \\ \left(\sum_k \delta_k^{(\ell+1)} W_{j,k}^{(\ell+1)} \right) g'(a_j^{(\ell)}), & \text{hidden unit (recursive)} \end{cases}$$



δ for a hidden unit = weighted sum of downstream δ s, modulated by g' .

Backprop in matrix form

Stack layer activations into vectors. Let $\delta_\ell = \partial E / \partial z^{(\ell)}$ and W_ℓ be the weight matrix.

Algorithm.

1. Initialize weights W_ℓ for every layer.
2. **Forward:** push x through, cache $z^{(1)}, z^{(2)}, \dots$
3. **Output δ :** set $\delta_n = \partial E / \partial z^{(n)}$.
4. **Backward sweep,** for $\ell = n, n-1, \dots, 1$:

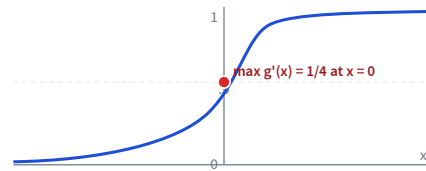
$$\delta_{\ell-1} = \delta_\ell \cdot \frac{\partial g(x^{(\ell)})}{\partial x^{(\ell)}} \cdot W_\ell$$
$$\frac{\partial E}{\partial W_\ell} = \delta_\ell \cdot \frac{\partial g(x^{(\ell)})}{\partial x^{(\ell)}} \cdot z^{(\ell-1)}$$

5. Plug $\partial E / \partial W_\ell$ into gradient descent.

The sigmoid derivative trick

$$\text{For } g(x) = \frac{1}{1 + e^{-x}}:$$

$$g'(x) = \frac{1}{1 + e^{-x}} \cdot \frac{e^{-x}}{1 + e^{-x}} = g(x) (1 - g(x))$$



Why it matters: during the forward pass we already compute $g(a)$. Reuse it — no need to redo the exponential during the backward pass.

Similar shortcuts exist for $\tanh(1 - g^2)$ and ReLU (0 or 1) — ReLU's constant gradient is a big reason it beats sigmoid in deep nets.

Beyond plain SGD: momentum and Adam

Plain SGD is noisy and slow in ravines. Two ideas fix it:

Momentum

Accumulate a velocity (an EWMA of past gradients) so consistent directions build speed:

$$v \leftarrow \beta v - \eta \nabla_W L, \quad W \leftarrow W + v$$

Adam

Per-parameter step sizes: divide by a running estimate of each gradient's magnitude, plus momentum. Robust default:

$$W \leftarrow W - \eta \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}$$

AdamW (Adam + decoupled weight decay) is the standard optimizer for transformers/LLMs today. [\[optional deep-dive deck\]](#)

Making training generalize & stay stable

Regularization (fight overfitting)

- **Weight decay:** add $\lambda \|W\|^2$ to the loss (L16).
- **Dropout:** randomly zero some units each step so the net can't rely on any one.
- **Early stopping:** halt when validation loss turns up.

Normalization & schedules (stay stable)

- **Batch / layer norm:** re-center & scale activations so gradients stay well-behaved.
- **LR warmup + cosine decay:** ramp η up, then anneal it down.
- **Gradient clipping:** cap the gradient norm to avoid blow-ups.

These are the everyday knobs behind training any modern deep network.

When to use (or not use) a neural network

Reach for an NN when

- High-dimensional, real-valued, or noisy inputs.
- Target function form is unknown (no good hand-crafted model).
- Interpretability is not a priority.
- Plenty of training data is available.

Avoid an NN when

- Architecture is hard to choose (layers, units, activations).
- Weights need to be inspected and explained.
- Data is tabular and small — overfitting risk is high.

Neural network vs decision tree

	Neural network	Decision tree
Data type	Images, audio, text	Tabular data (gradient boosting usually wins here)
Data size	Needs a lot; easy to overfit small data	Works with very little data
Target function	Arbitrary functions	Nested if/else rules
Architecture	Layers, units, activations, init, η all critical	A few hyperparameters (depth, pruning)
Interpretability	Black box	Easy to explain to humans
Train / inference	Slow to train and run	Fast

Revisiting learning goals

- Write the **cross-entropy** loss and the gradient-descent update.
- Describe **back-propagation** and compute a weight's gradient.
- Speed up training with **momentum** and **Adam / AdamW**.
- Fight overfitting with **regularization, dropout, and normalization**.
- Decide when to use a **neural network** vs a **decision tree**.

Next: Transformers & Attention

We can now build and train deep nets. But how do we handle **sequences** — language, code, audio — at scale?

- Why RNNs struggle with long sequences.
- **Self-attention**: the idea behind every modern LLM.
- The **transformer** block, end to end.

L21: Transformers & Attention.